

Apartment Rental Management System

Mohammed Khalaf, David Ulloa

Department of Engineering and Computer Science, California State University, Fullerton

CPSC 332: File Structures and Database Systems

Professor Ken Wuttisasiwat

Mar 26, 2026

System Description

The proposed Apartment Rental Management System is a relational database designed to support the daily operations of residential rental properties. Apartment managers and leasing offices must manage connected information involving tenants, apartment units, lease agreements, rent payments, and maintenance activity. When that information is handled through paper files or disconnected spreadsheets, it becomes more difficult to maintain accurate records and monitor ongoing business activity. This system centralizes those records in one structured database so that property information can be stored consistently, updated efficiently, and reviewed more easily.

The primary users of the system will be property managers, leasing staff, maintenance staff, and property administrators. Leasing staff will use the system to create tenant records, assign units, and manage lease data. Property managers will use it to review occupancy, payment activity, and lease status across one or more properties. Maintenance staff will use it to monitor service requests tied to specific units. The system will track both data and workflow, including properties, units, tenants, leases, payments, maintenance requests, and status values. A typical workflow begins when a tenant signs a lease, continues through rent collection and maintenance support during occupancy, and ends when the lease is renewed, expired, or terminated.

Scope:

- The system will store tenant information, including name, email, phone number, and tenant identifiers.
- The system will store property information, including property name, address, and

management details.

- The system will store unit information, including unit number, monthly rent, bedroom count, and occupancy details.
- The system will create and manage lease agreements that connect tenants to specific apartment units.
- The system will track lease dates, security deposits, and lease status values such as Active, Expired, Renewed, or Terminated.
- The system will record rent payments associated with a specific lease and preserve payment history over time.
- The system will create and manage maintenance requests for apartment units and track request progress.
- The system will help prevent invalid activity, such as overlapping active leases for the same unit or payments without a related lease.

UML Overview (High-Level)

The proposed database includes a total of 9 tables which are six core tables. Two lookup/reference tables, and one junction table to resolve a many-to-many relationship.

Entities (Tables) With Example Attributes

- **Tenants** (tenant_id, full_name)
- **Properties** (property_id, property_name)
- **Units** (unit_id, unit_number)

- **Leases** (lease_id, start_date)
- **Payments** (payment_id, payment_date)
- **Maintenance_Requests** (request_id, request_date)
- **Lease_Status** (status_id, status_name)
- **Payment_Status** (payments_status_id, payment_status_name)
- **Lease_Tenants** (lease_id, tenant_id)

Core Tables

The core tables are **Tenants**, **Properties**, **Units**, **Leases**, **Payments**, and **Maintenance_Requests**. These tables store the main operational data needed to manage occupancy, lease activity, payment records, and maintenance support.

Lookup / Reference Tables

The lookup tables are **Lease_Status** and **Payment_Status**. These tables standardize status values and improve consistency by limiting entries to approved values.

Relationships With Cardinality

- **Properties to Units**: (1:N) because one property can contain many units, but each unit belongs to one property.
- **Units to Leases**: (1:N) because one unit can have many leases over time, but each lease is associated with one unit.
- **Leases to Payments**: (1:N) because one lease can generate many payments, but each

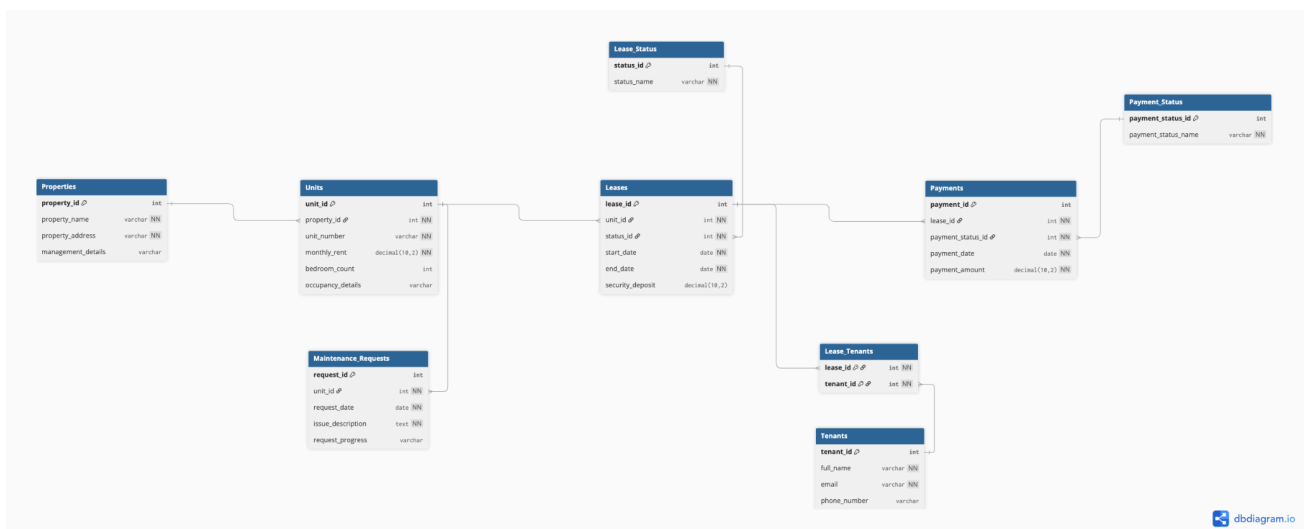
payment belongs to one lease .

- **Units to Maintenance_Requests:** (1:N) because one unit can have many maintenance requests, but each request belongs to one unit.
- **Lease_Status to Leases:** (1:N) because one status can apply to many leases, but each lease has one current status.
- **Payment_Status to Payments:** (1:N) because one status can apply to many payments, but each payment has one current status.

Many-to-Many Relationship and Junction Table

- **Tenants to Leases:** (M:M) because one tenant can be associated with multiple leases over time, and one lease can include multiple tenants, such as roommates or family members. This many-to-many relationship is resolved through the **Lease_Tenants** junction table.

UML Diagram:



The UML diagram shows the entities and relationships in the proposed database system. It demonstrates one to many relationships between properties and units, units and leases, leases and payments, and units and maintenance requests along with the many-to-many relationship between tenants and leases that is resolved through the Lease_Tenants table.

Business Rules / Constraints

- Each tenant must have a unique tenant_id.
- Each tenant email must be unique.
- Each property must have a unique property_id.
- Each unit must belong to exactly one property.
- A lease cannot exist without a valid unit record.
- A payment cannot exist without a valid lease record.
- A maintenance request cannot exist without a valid unit record.
- The lease start_date must occur before the lease end_date.
- A unit cannot have more than one active lease for the same time period.
- A lease status must come from the predefined values stored in Lease_Status.
- A payment status must come from the predefined values stored in Payment_Status.
- Each tenant-to-lease combination in Lease_Tenants must be unique.